

Mikrocomputertechnik 1

Einheit 6: Das Steuerwerk

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 84

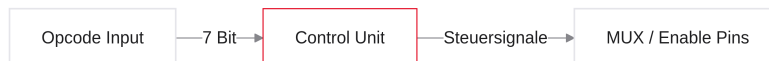
Das Steuerwerk

Folie

2 / 84

Die Control Unit als Dirigent der Hardware

- Herzlich willkommen zur sechsten Vorlesung in Mikrocomputertechnik 1
- Letzte Einheit: das physikalische Rohrleitungssystem (Datenpfad)
- Multiplexer verlegt, ALU verkabelt, Speichermodule angeschlossen
- Das System ist ohne Steuersignale noch leblos
- Heute: das Gehirn der CPU — die Control Unit (das Steuerwerk)

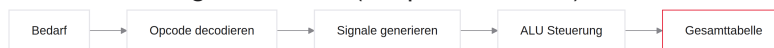


Notizen

Herzlich willkommen zu Einheit 6. In der letzten Einheit haben wir ein Rohrleitungssystem aus Multiplexern, ALU und Speicher verlegt. Ohne Steuersignale wissen Multiplexer nicht, ob sie auf 0 oder 1 schalten sollen; das Register File weiß nicht, ob es schreiben darf. Heute wecken wir den Prozessor zum Leben: wir entwerfen das Steuerwerk, die Control Unit.

Agenda für den heutigen Tag

1. Das blinde System — warum Hardware ohne Dirigent kollabiert
2. Befehlsdecodierung — wie der Chip den Opcode interpretiert
3. Die Haupt-Steuersignale — Hebel für den gesamten Datenpfad
4. ALU Control — zweistufige Decodierung (`funct3` / `funct7`)
5. Signal-Trace — die zentrale Wahrheitstabelle ausfüllen
6. Ripes-Praxis — Vorhersage-Methode („Paper & Pencil“) im Simulator



Notizen

Die Agenda führt von der Notwendigkeit eines Steuerwerks zur vollständigen Wahrheitstabelle. Zuerst identifizieren wir Steuersignale, dann sezieren wir das 32-Bit-Befehlsword und den Opcode. Anschließend legen wir für Load, Store, Branch und R-Type die Schalterstellungen fest und übertragen die Theorie in eine Vorhersage-Methode für Ripes.

Folie

Rückblick: Der fertige Datenpfad

- Endergebnis der letzten Vorlesung (Patterson/Hennessy, vereinfacht)
- Register, Speicher, ALU und drei Multiplexer
- Theoretisch ausführbar: R-Type, Load, Store, Branch
- Datenkabel (meist 32 Bit) sind fest verdrahtet
- Fehlendes Puzzleteil: die Steuerleitungen „von oben“

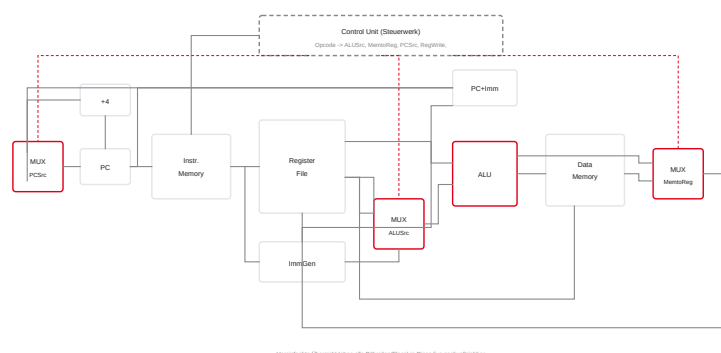


Abbildung 1: Single-Cycle-Datenpfad (vereinfacht)

Notizen

Der Datenpfad steht. Der Program Counter zieht die Instruktion, Registeradressen laufen ins Register File,

Daten fließen in ALU und RAM. Achten Sie auf die Steuerleitungen zu Multiplexern und Enable-Pins — das ist die Arbeit von heute.

Folie

5 / 84

Der Datenpfad ist blind

- Kein Bauteil weiß, welchen Befehl Sie in C oder Assembler getippt haben
- Die ALU hat keine Augen für `add t0, t1, t2`
- Ein Multiplexer schaltet nicht, weil er einen `lw`-Befehl „merkt“
- Jedes Bauteil wartet auf eine physikalische Spannung (1 oder 0)
- Fehlt diese Spannung — oder ist sie falsch — kollabiert das System

Ein Multiplexer ohne Steuersignal:

Ist das Eingangssignal 0 oder 1?

-> Ohne elektrische Vorgabe: undefinierter Zustand

-> Daten kollidieren, der Prozessor friert ein

Notizen

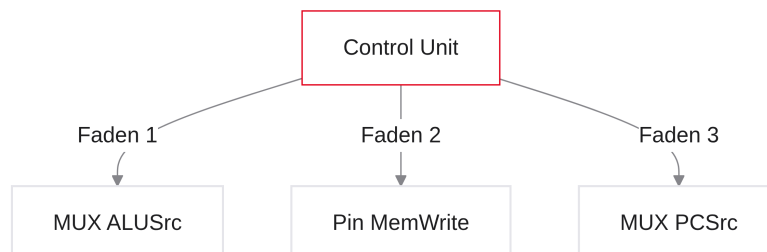
Zentrale Erkenntnis: Der Datenpfad ist blind. Ein Multiplexer kennt keinen Code — nur ein Steuerkabel mit Strom an (1) oder aus (0). Ohne aktives Setzen wählt der MUX undefiniert; ALU und RAM arbeiten mit Müll.

Folie

6 / 84

Die Marionetten-Metapher

- Datenpfad als Marionette
- Gelenke: Multiplexer und Write-Enable-Pins
- Arme und Beine bewegen sich nicht von allein
- Puppenspieler: die Control Unit
- Kupferfäden: Steuerleitungen — im Takt gezogen, tanzt die Puppe



Notizen

Marionetten-Metapher: Der Datenpfad ist der Körper, die Multiplexer sind Gelenke. Das Steuerwerk hält

die Kupferfäden. Bei einer Addition: ALUSrc auf 0, RegWrite aktiv, MemWrite inaktiv — die Puppe vollführt den richtigen Tanz.

Folie

7 / 84

Was ist die Control Unit?

- Separate, fest verdrahtete Box aus Logikgattern auf dem Chip
- In Architekturdiagrammen oft als Wolke oder abgerundete Box
- Input: Teile des aktuellen 32-Bit-Befehlswords
- Verarbeitung: reine Kombinatorik (AND/OR-Netzwerke)
- Output: 1-Bit- bzw. 2-Bit-Steuersignale an den Datenpfad

Notizen

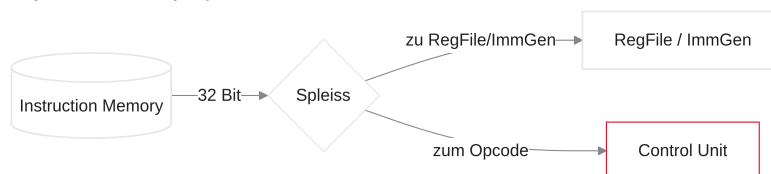
Physikalisch ist das Steuerwerk keine Magie, sondern Logikgatter — wie in Einheit 1. Es empfängt das Befehlsword, jagt es durch Gatter und liefert an den Ausgangspins die Spannungspegel für Multiplexer und Enable-Pins.

Folie

8 / 84

Input: Woher kommen die Befehle?

- Der Program Counter adressiert den Befehlsspeicher (Instruction Memory)
- Der Speicher liefert 32 Bit (Phase: Instruction Fetch)
- Kabelspleiß: ein Teil wandert in den Datenpfad (Register, ImmGen)
- Ein anderer Teil wird abgezweigt in die Control Unit
- Der Puppenspieler liest physikalisch mit, welcher Code aus dem ROM kommt



Notizen

Das 32-Bit-Kabel aus dem Befehlsspeicher wird aufgespleißt: ein Teil liefert Registeradressen und Immediate, ein Teil geht nach unten in die Control Unit. Sie liest den Maschinencode in Echtzeit mit.

Output: Die blauen Steuerkabel

- Auf der Ausgangsseite treten die Steuerleitungen aus
- Für unseren RV32I-Single-Cycle-Prozessor: sieben primäre Steuersignale
- Die meisten Signale sind 1 Bit breit (Strom an = 1, aus = 0)

Kategorie	Signale
Multiplexer (Weichen)	ALUSrc, MemtoReg, Branch (für PCSrc)
Erlaubnis (Write-Enable)	RegWrite, MemWrite
Leseschutz	MemRead
Sondertyp (2 Bit)	ALUOp (grobe ALU-Steuerung)

Notizen

Sieben Steuersignale dirigieren den Datenpfad: drei für Multiplexer (ALUSrc, MemtoReg, Branch), zwei Write-Enables (RegWrite, MemWrite), MemRead für den RAM und ALUOp zur ALU. Achtung bei der Zählung: ALUOp ist 2 Bit breit — wir sprechen von sieben Signalen, aber acht Steuerbits auf dem Chip. Das ist unser Instrumentarium.

Kombinatorische Logik (Das Timing)

- Wichtige Eigenschaft: Die Control Unit hat keinen eigenen Takt
- Reine kombinatorische Schaltung (wie ein großer Addierer)
- Kein Zustand, keine Flip-Flops, kein Speicher
- Nach Gatterlaufzeit (wenige Nanosekunden) stehen die Signale fest
- Der Datenpfad nutzt sie im Taktzyklus

```
/* Control Unit: rein funktional, kein State */
Steuersignale = generate_control(Instruction_Bits);
```

Notizen

Hat die Control Unit einen Takt? Nein. Sie ist ein Durchlaufnetzwerk aus Gattern: Strom hinein, hindurch, hinaus — ohne Gedächtnis. Sobald das Befehlswort anliegt, stehen die Steuersignale bereit, oft lange bevor die Daten die ALU erreichen.

Die Wahrheitstabelle (Refresher)

- Kombinatorik lässt sich vollständig als Wahrheitstabelle beschreiben
- Input-Spalten: Bit-Kombinationen der Eingabe
- Output-Spalten: resultierende Steuersignale
- Die Wahrheitstabelle ist der Bauplan für den Hardware-Entwurf
- Heutige Mission: diese Matrix für unseren Prozessor ausfüllen

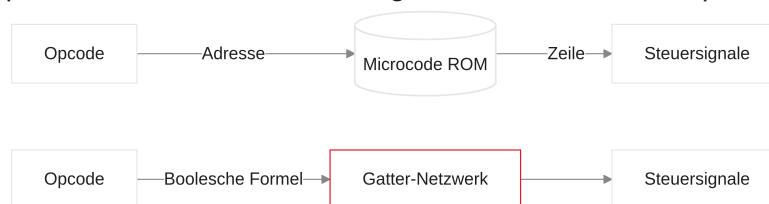
Input A	Input B	Output Y (AND)	Output Z (XOR)
0	0	0	0
0	1	0	1
...	...	...	...

Notizen

Kreis zu Einheit 1: AND, OR, XOR. Ohne Gedächtnis bestimmt allein der aktuelle Eingang das Verhalten — abbildbar als Wahrheitstabelle. Kennen wir die Bits für ADD, legen wir fest, welche Ausgänge 1 bzw. 0 sein müssen. Daraus entstehen die physischen Gatter.

Architektur-Design: LUT vs. Gatter

- Methode 1: Hardwired Logic — Karnaugh-Veitch, minimierte AND/OR-Gatter; schnell, aber starr
- Methode 2: Microcode (ROM / LUT) — Wahrheitstabelle als Look-up; Opcode als Adresse
- RISC-V ist prädestiniert für Hardwired Logic: kleine, effiziente Chips



Notizen

Zwei Philosophien: Hardwired Logic gießt die Matrix in Gatter — schnell und starr. Microcode schreibt die Tabelle in einen internen ROM; der Opcode ist die Zeilennummer. Bug im Hardwired-Design heißt oft: Chip neu fertigen.

Der RISC-V-Vorteil

- x86 (CISC): hunderte komplexe Befehle, teils bis 15 Bytes
- Hardwired-Design dort praktisch unmöglich — große Microcode-ROMs
- RISC-V: uniforme 32-Bit-Befehle
- Steuerwerk oft mit wenigen Dutzend Logikgattern hardwired
- Vorteil: Strom, Chipfläche und Herstellungskosten

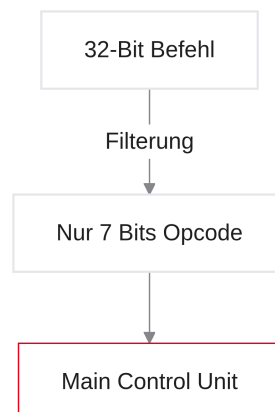
Notizen

RISC-Philosophie aus Einheit 2: CISC braucht riesige Microcode-Tabellen. RISC-V ist so entworfen, dass Steuersignale mit einer Handvoll Gatter ausrechenbar sind — die Control Unit bleibt winzig.

Befehlsdecodierung und Opcode

Wir müssen den Befehl lesen — aber wie?

- Die Control Unit muss den Befehl interpretieren
- Müssen wirklich alle 32 Bits gleichzeitig analysiert werden?
- Antwort liefert die Anatomie der RISC-V-Formate
- Der 7-Bit-`opcode` als Identifikationsschild
- Das Konzept Don't Care (X) und warum es Hardware spart



Notizen

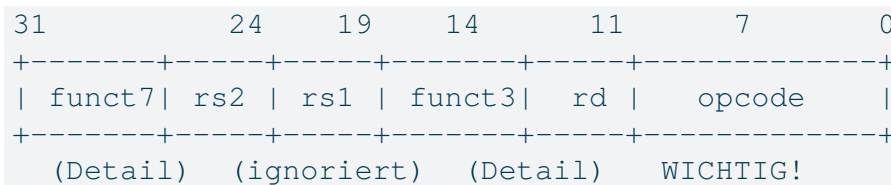
Phase 2: Wahrheitstabelle konstruieren. 32 Eingangsbits wären 2^{32} Zeilen — unmöglich. Welche Bits sind wirklich entscheidend?

Folie

16 / 84

Die Anatomie des Befehls (Refresher)

- Instruction Formats R, I, S, B aus Einheit 2
- Felder wie `rs1` (Bits 19:15) oder `rd` (Bits 11:7) sind Registeradressen
- Die Control Unit interessiert sich nicht für diese Hausnummern
- Ob `t0` oder `s11` addiert wird: MUX-Stellungen bleiben gleich
- Diese Bits können wir für das Steuerwerk ignorieren



Notizen

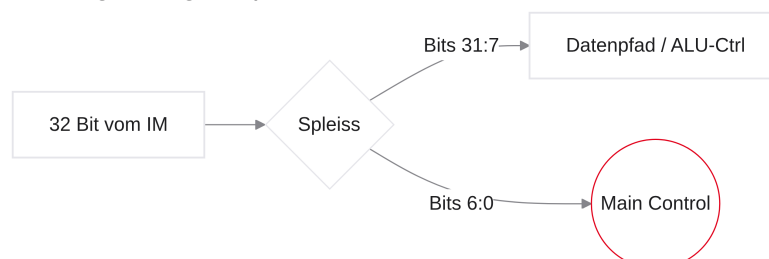
Gut 15 Bits sind reine Registeradressen. Ob Fach 5 mit Fach 10 oder Fach 31 mit Fach 1 addiert wird: der physikalische Ablauf bleibt derselbe. Wir filtern radikal.

Folie

17 / 84

Das 7-Bit-Identifikationsschild

- In jedem Format an derselben Stelle: der `opcode`
- Bits `[6 : 0]` — exakt 7 Bit breit
- Diese 7 Drähte sind der Input der Main Control Unit
- Der Opcode sagt die Befehlsfamilie, nicht `add` vs. `sub`
- Detailentscheidungen folgen später über `funct3` / `funct7`



Notizen

Sieben Bits rechts: $2^7 = 128$ mögliche Zeilen — überschaubar. Der Opcode verrät die grobe Familie; Feinheit kommt über eine zweite Stufe (ALU Control).

Folie

18/84

Opcode: R-Type (Rechnen)

- Familie: zwei Register lesen, rechnen, in ein Register schreiben (add, sub, and, or, sll, ...)
- Opcode (binär): 0110011
- Keine Konstanten nötig; RAM unbeteiligt; RegWrite zwingend

Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	1	1	0	0	1	1

Notizen

Sieht die Control Unit 0110011: R-Type. ALUSrc auf Register, ImmGen arbeitslos, RAM unnötig, RegWrite am Ende aktiv.

Folie

19/84

Opcode: I-Type (Load)

- Familie: Daten aus dem Arbeitsspeicher in die CPU (lw, lb, lh, ...)
- Opcode (binär): 0000011
- Immediate für Adressberechnung; RAM lesen; Write-Back aus dem RAM

Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	0	0	0	0	1	1

Notizen

Load: ALUSrc lässt die Konstante durch, MemtoReg nimmt den RAM-Pfad, MemRead = 1.

Opcode: S-Type (Store)

- Familie: CPU-Daten im RAM ablegen (sw, sb, sh)
- Opcode (binär): 0100011
- Immediate für Adresse; MemWrite aktiv; kein rd $\rightarrow$ RegWrite = 0

Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	1	0	0	0	1	1

Notizen

Store ist sensibel: MemWrite = 1, RegWrite = 0 — sonst landet Müll im Register File.

Opcode: B-Type (Branch)

- Familie: bedingte Sprünge (beq, bne, blt, ...)
- Opcode (binär): 1100011
- Registervergleich; nichts schreiben; Sprungentscheidung mit ALU-Flag

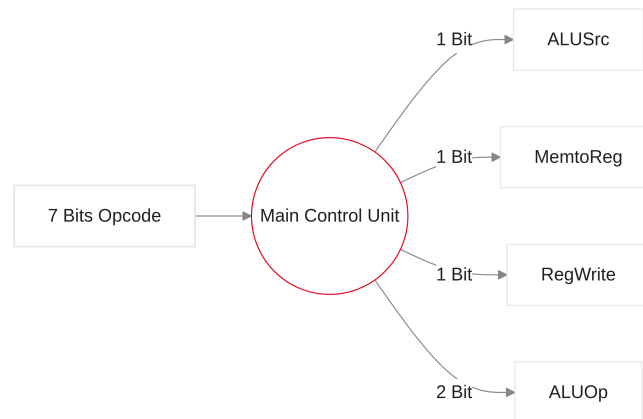
Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
1	1	0	0	0	1	1

Notizen

Branch: RegWrite und MemWrite bleiben 0. Das Branch-Signal bereitet das AND-Gatter vor dem PC vor — bei Zero-Flag kann gesprungen werden.

Der Main-Control-Block

- Blackbox konkret: exakt 7 Bit Eingang
- Ausgang: Bündel an Kontrollsignalen
- Signale fließen zu Multiplexern und Speicher-Pins
- Nächste Aufgabe: Wahrheitstabelle Zeile für Zeile lösen



Notizen

Mathematisch eingerahmt: Main Control mit 7 Eingängen und mehreren Ausgängen. Pro Opcode festlegen, wo 1 und wo 0 anliegt — dann ist das Gehirn spezifiziert.

Das Konzept Don't Care (X)

- Ingenieurswerkzeug: Don't Care (X)
- Beispiel: MemtoReg bei sw — Store schreibt nie ins Register File (RegWrite = 0)
- Welches Kabel der MUX durchlässt, ist irrelevant
- In die Wahrheitstabelle: X statt 0 oder 1

```

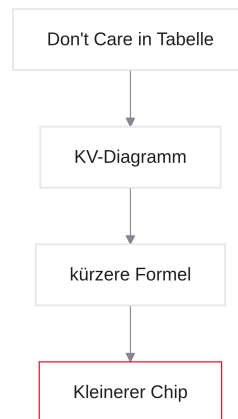
MemtoReg bei sw:
Muss es 0 sein (ALU)? -> Nein
Muss es 1 sein (RAM)? -> Nein
Also: X (Don't Care)
  
```

Notizen

Bei Branch oder Store ist RegWrite = 0; die Stellung von MemtoReg ist egal. X heißt: Synthese darf wählen, was günstiger ist.

Warum X Hardware spart

- X gibt dem Synthese-Tool (Quartus, Vivado) Freiheit
- Im Karnaugh-Veitch-Diagramm: X wahlweise als 1 oder 0 für große Blöcke
- Große Blöcke → kurze Boolesche Formeln
- Kurze Formeln → weniger Gatter, weniger Strom, höhere Taktraten



Notizen

Sture Nullen kosten Transistoren. X ist ein Joker für die Minimierung: größere KV-Blöcke, weniger Gatter, billigere und schnellere Chips.

Die Skelett-Tabelle

- Leere Wahrheitstabelle als Arbeitsblatt
- Vier Zeilen für die wichtigsten Opcodes
- Spalten für die entscheidenden Steuersignale
- Nächste Folien: Matrix durch logisches Denken füllen

Instruktion (Opcode)	ALUS- rc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
R-Type (0110011)	?	?	?	?	?	?	?	?
lw (0000011)	?	?	?	?	?	?	?	?
sw (0100011)	?	?	?	?	?	?	?	?
beq (1100011)	?	?	?	?	?	?	?	?

Notizen

Links die Opcodes, rechts die Ausgänge. Niemand lernt die Tabelle auswendig — sie lässt sich aus dem Datenpfad der letzten Einheit herleiten.

Folie

26 / 84

Die Spalten gruppieren

- Output-Pins mental nach CPU-Phase ordnen
- Execute (ALU): `ALUSrc`; `ALUOp1/ALUOp0` für die grobe ALU-Aufgabe
- Memory (RAM): `MemRead`, `MemWrite`; `Branch` für die Sprung-Erlaubnis
- Write-Back: `MemtoReg` und `RegWrite`

Signalfluss in der CPU: von links nach rechts.
Tabelle gedanklich in derselben Richtung ausfüllen.

Notizen

Struktur: `ALUSrc` in Phase EX, `MemRead/MemWrite` in MEM, `MemtoReg/RegWrite` in WB. Den Pfad von links nach rechts durchgehen.

Folie

27 / 84

Das eiserne Gesetz der Präzision

- Ein Fehler in der Matrix kann das System crashen lassen
- Beispiel: `RegWrite = 0` bei `sw` vergessen
- Zufälliges Zielregister (z. B. `s0`) wird mit Müll überschrieben
- Unvorhersehbare Variablenzerstörung — klassischer Heisenbug
- Chip-Verifikation ist deshalb ein Milliardenmarkt

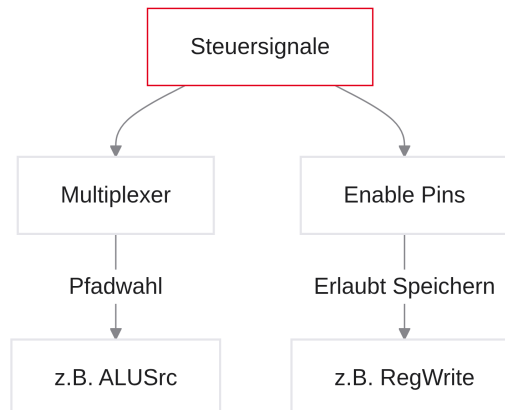
Notizen

Hier schreiben wir keine Software mit Patch. Eine falsche Null in der Tabelle kostet Wafer. Deshalb verifizieren Konzerne diese Matrizen intensiv, bevor Silizium gefertigt wird.

Die Haupt-Steuersignale

Die Signale im Überblick

- Jedes der sieben Steuersignale systematisch durchgehen
- Welche Signale sind Weichen (MUX)?
- Welche Signale sind Schreib-/Leserechte (Enable-Pins)?
- Warum sind Don't Cares ($\bar{X}$) bei MUXen oft sinnvoll?
- Die fatale Kollision von MemRead und MemWrite

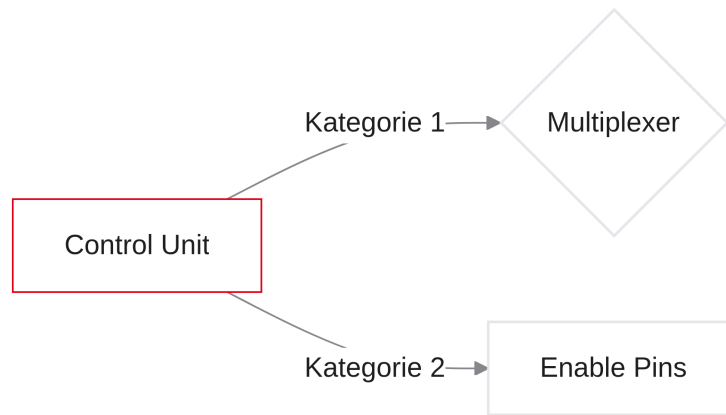


Notizen

Phase 3 beginnt: jeden Pin des Patterson/Hennessy-Datenpfads noch einmal schärfen, bevor die Matrix final ausgefüllt wird.

Zwei Arten von Signalen

- Im Datenpfad gibt es zwei Kategorien von Steuerleitungen
- Kategorie 1: Weichen (Multiplexer) — lenken Datenströme (z. B. ALUSrc)
- Eine Weiche leitet immer weiter — auch bei Müll; sie steht auf 0 oder 1
- Kategorie 2: Erlaubnis-Pins (Write-Enable) — verändern Speicherzustand (z. B. RegWrite)
- Enable-Pins restriktiv auf 0 halten; nur bei Bedarf 1 — sonst Datenkorruption



Notizen

Bevor wir die Tabelle befüllen: Multiplexer schalten links oder rechts durch und blockieren nie komplett — am Ausgang liegt immer Strom, notfalls Müll. Das ist ungefährlich, solange wir ihn nicht speichern. Enable-Pins sind die Türsteher an RAM und Register File: nur bei 1 wird geschrieben.

Folie

31 / 84

ALUSrc (Multiplexer 1)

- 1-Bit-Signal steuert den MUX vor dem zweiten ALU-Eingang
- 0: ALU rechnet mit Register RD2 aus dem Register File
- 1: ALU rechnet mit der 32-Bit-Konstante aus dem ImmGen
- Reine Registeroperationen (add, sub, and) → $ALUSrc = 0$
- Adress-Offsets (lw, sw) → $ALUSrc = 1$

```

ALUSrc:
R-Type  -> 0    (rs2)
Load     -> 1    (Immediate-Offset)
Store    -> 1    (Immediate-Offset)
Branch   -> 0    (Register vergleichen)
  
```

Notizen

Bei ADD sendet die Control Unit 0 (Registerkabel). Bei Load/Store sendet sie 1 (Base + Immediate). Bei Branch müssen zwei Register verglichen werden — wieder 0.

MemtoReg (Multiplexer 2)

- 1-Bit-Signal steuert den MUX vor dem Write-Back
- 0: ALU-Ergebnis fließt ins Zielregister
- 1: RAM-Daten fließen ins Zielregister
- RAM-Daten nur beim Load (`lw`)
- Bei R-Type muss `MemtoReg = 0` sein — sonst RAM-Schrott in der Variablen

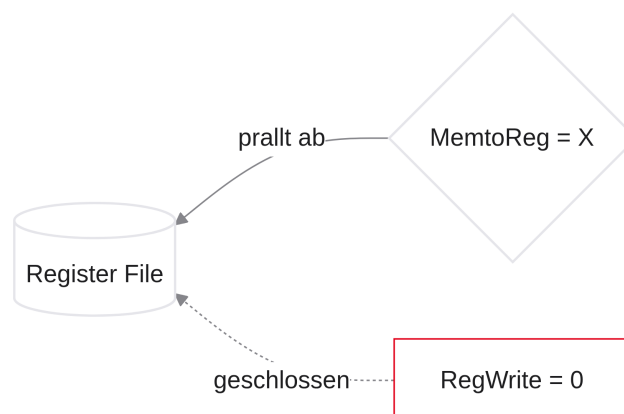
```
MemtoReg:
R-Type  -> 0    (ALU-Ergebnis)
Load    -> 1    (RAM-Daten)
Branch  -> X    (kein Speichern)
Store    -> X    (kein Speichern)
```

Notizen

MemtoReg sitzt ganz rechts: ALU-Ergebnis oder RAM-Ergebnis auf die Werkbank. `ADD` speichert ALU (0), `lw` speichert RAM (1). Was ist mit Befehlen ohne Write-Back?

MemtoReg als Don't Care (*X*)

- Bei `sw` oder `beq`: nichts wird ins Register File geschrieben
- `RegWrite = 0` — Werkbank verschlossen
- MUX-Stellung vor dem Register File ist irrelevant
- Signale prallen am geschlossenen Register File ab
- In die Wahrheitstabelle: stolzes *X*



Notizen

Erstes Don't Care: Bei Store Word gibt es kein `rd` (Immediate gesplittet). `RegWrite` ist 0 — `MemtoReg`-

Folie

34 / 84

RegWrite (Erlaubnis-Pin)

- Hochkritischer Write-Enable am Register File
- 1: Bei der nächsten positiven Taktflanke wird `rd` überschrieben
- 0: Register File schreibgeschützt
- Fehler hier sind für die Software katastrophal

```
RegWrite:
R-Type  -> 1    (Ergebnis speichern)
Load     -> 1    (geladenes Wort speichern)
Store    -> 0
Branch   -> 0
```

Notizen

RegWrite ist der Wächter der 32 Architekturregister. Bei R-Type und Load auf 1, weil beide aktiv nach `rd` schreiben.

Folie

35 / 84

Wer darf nicht schreiben?

- Zwei Familien dürfen `RegWrite` nie aktivieren
- Store (`sw`): Ziel ist der RAM; CPU-Variablen bleiben unangetastet
- Branch (`beq`): Ziel ist der PC; keine Datenberechnung/-ladung
- Control Unit zwingt `RegWrite` eisern auf 0
- Ohne dieses Signal würde `sw` ein zufälliges Register mit Schrott beschreiben

Notizen

Branch vergleicht und manipuliert den PC; Store schreibt in den RAM. In beiden Fällen darf das Register File nicht verändert werden.

MemRead (Erlaubnis-Pin)

- Erlaubnis-Pin für das Lesen aus dem Datenspeicher (RAM)
- 1: RAM legt Bits an der ALU-Adresse auf das Lesekabel
- 0: Ausgang neutral / blockiert (hochohmig)
- Im Single-Cycle nur bei `lw` auf 1
- Sonst Strom sparen: RAM nicht unnötig auslesen

```
MemRead:  
Load          -> 1  
Alle anderen -> 0
```

Notizen

Bei ADD ist das ALU-Ergebnis keine RAM-Adresse. Ständiges Auslesen würde Strom verschwenden. MemRead nur bei Load. Hinweis: In manchen Patterson/Hennessy-Varianten ist MemRead weniger prominent oder implizit. Wir führen es explizit wie in Einheit 5 und in Ripes — als Lehrbuch-/Simulator-Signal für Lesefreigabe und Stromspar-Logik.

MemWrite (Erlaubnis-Pin)

- Noch kritischerer Write-Enable am RAM
- 1: Bei Taktflanke wird Write-Data in die RAM-Zelle geschrieben
- 0: Schreibschutz aktiv
- In der Basistabelle genau einmal 1: bei `sw`
- `MemWrite = 1` bei anderen Befehlen zerstört Speicher und oft das Betriebssystem

```
MemWrite:  
Store          -> 1  
Alle anderen -> 0
```

Notizen

MemWrite ist der Tresorschlüssel. Nur bei Opcode `0100011` (Store) auf 1; sonst eiserne 0.

Die Gefahr: MemRead und MemWrite

- Logikfehler könnte beide Signale gleichzeitig auf 1 setzen
- Gleichzeitig schreiben und lesen auf denselben Leitungen
- Risiko: Kurzschluss / Datenkorruption auf Chip-Ebene
- Unsere Wahrheitstabelle: nie beide zugleich 1
- Entweder laden oder speichern — nicht beides

Notizen

Viele Speicherchips teilen Lese-/Schreibpfade. MemRead und MemWrite gleichzeitig kann Sense Amps zerstören. In der Matrix ist immer höchstens eines aktiv.

Branch (Entscheidungs-Signal)

- Weder direkter MUX noch Enable-Pin
- Hilfssignal in das AND-Gatter vor dem PC
- 1: Control Unit bestätigt — aktueller Befehl ist bedingter Sprung
- Liefert dem AND die halbe Erlaubnis für `PCSrc`
- Bei `add`, `lw`, `sw`: Branch = 0 — Sprung unmöglich

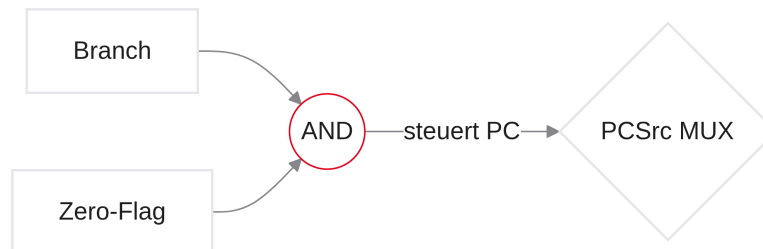
```
Branch:  
beq / Branch-Familie -> 1  
Alle anderen         -> 0
```

Notizen

Branch ist ein Stempel der Control Unit: „Ja, das ist ein Sprungbefehl.“ Er fließt ins AND vor dem `PCSrc`-MUX.

Die PCSrc-Logik (Rückblick)

- Control Unit kann die finale Sprungentscheidung nicht allein fällen
- Registervergleich fehlt ihr — ALU-Zero als Assistent
- Für `beq`: $\text{PCSrc} = \text{Branch} \wedge \text{Zero}$
- Beide Signale müssen 1 sein, damit der PC umgeschaltet wird
- Andere Branches (z. B. `bne`): andere Flag-Logik (oft $\text{Branch} \wedge \overline{\text{Zero}}$)



Notizen

Die Control Unit liest nur den Opcode: sie weiß, dass ein Branch vorliegt, nicht ob die Registerbedingung gilt. Für `beq` gilt: $\text{PCSrc} = \text{Branch} \text{ AND } \text{Zero}$. Für `bne` typischerweise $\text{Branch} \text{ AND NOT Zero}$.

Zwischenstand der Tabelle

- Weichen und Enable-Pins für den Chip unter Kontrolle
- Datenverkehr und Speicherblockaden sind spezifiziert
- Noch offen: Was genau soll die ALU tun?
- Main Control hat bisher nur die groben Pfade organisiert

Signal	Funktion	Bauteil
ALUSrc	0=Reg, 1=Imm	MUX vor ALU
MemtoReg	0=ALU, 1=RAM	MUX vor Write-Back
RegWrite	1=Schreiben	Register-File-Pin
MemRead/MemWrite	1=Aktiv	RAM-Pins
Branch	1=Sprungbereit	AND vor PC

Notizen

Sechs von sieben Schaltern sind klar. Offen bleibt: addieren, subtrahieren oder AND?

Das fehlende ALU-Signal

- Warum kein simples `ALU_Addition`-Signal in der Main Control?
- Problem: der 7-Bit-Opcode
- `add` und `sub` teilen denselben Opcode `0110011`
- Main Control sieht nur die Familie — nicht Plus vs. Minus

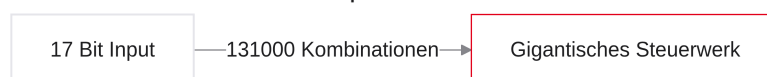
```
; Main Control sieht nur 0110011:  
add t0, t1, t2  
sub t0, t1, t2  
and t0, t1, t2
```

Notizen

Genialität und Hindernis der RISC-V-Codierung: `add`, `sub`, `and`, `or` haben denselben Opcode. Die Feininformation steckt nicht in Bits [6:0].

Das Problem mit R-Types

- Exakte Operation steckt in `funct3` und `funct7`
- Opcode + `funct3` + `funct7` = 17 Bit Input
- $2^{17} = 131\,072$ Zeilen Wahrheitstabelle
- Ein so großer Gatterblock wäre langsam und flächenhungrig
- Gegenteil der schnellen RISC-Philosophie



Notizen

17 Eingangsbits — 130.000+ Zeilen. Die Taktfrequenz würde einbrechen. Wir müssen klüger delegieren.

ALU Control (Zweistufige Decodierung)

Teile und Herrsche

- Ausweg aus der gigantischen Wahrheitstabelle
- Delegation von Verantwortung
- 2-Bit-Signal `ALUOp` als Brücke zwischen den Abteilungen
- Kleine ALU Control Unit als Detail-Spezialist
- Auswertung von `funct3` und `funct7`



Notizen

Phase 4: zweistufige Decodierung. Main Control sieht das grobe Bild und delegiert Details an eine zweite, winzige Einheit direkt vor der ALU.

Die Lösung: Zweistufige Kontrolle

- Logik in zwei separate Boxen
- Stufe 1 (Main Control): 7-Bit-Opcode; MUXe und Speicher; sendet `ALUOp`
- Stufe 2 (ALU Control): nimmt `ALUOp`, liest `funct3/funct7`, finales ALU-Kommando
- Zwei kleine Schaltungen sind schneller und kleiner als eine gigantische

Notizen

Main Control bleibt bei 128 Zeilen. Bei Bedarf funkt sie 2 Bit `ALUOp`. Die ALU Control bastelt daraus das konkrete Steuerwort für die Mathematik-Einheit.

Das Signal ALUOp (2 Bit)

- Verlässt die Main Control Richtung ALU Control
- 2 Bit — vier grobe Zustände
- Kategorisiert die Aufgabe der ALU

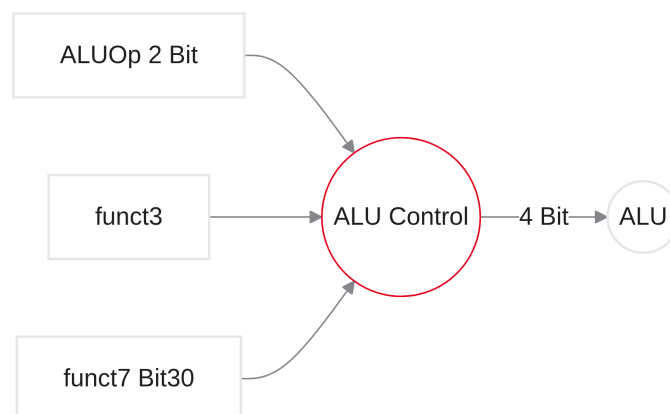
ALUOp	Bedeutung	Befehlstypen
00	ALU muss addieren (unbedingt)	lw, sw, addi
01	ALU muss subtrahieren (Vergleich)	beq
10	ALU muss rechnen (funct lesen)	R-Type
11	Reserve / ungenutzt	-

Notizen

Load/Store/addi → 00 (Addition ohne funct7). Branch → 01 (Subtraktion zum Vergleich). R-Type → 10 (funct-Felder). Bei addi darf nicht 10 stehen: Bit 30 wäre sonst Immediate statt funct7.

Die kleine ALU Control Unit

- Eingänge: ALUOp (2), funct3 (3), relevantes funct7-Bit (1)
- funct7 nur auswerten, wenn nötig (R-Type)
- Ausgang: 4-Bit-Steuerwort in die ALU
- Definiert die Verschaltung: Plus, Minus, AND, OR



Notizen

Winziges Gatter-Netzwerk: aus begrenztem Input das 4-Bit-Kommando für die ALU.

Fall 1: ALUOp = 00 (Load/Store)

- Chef funkt 00 — lw oder sw
- Hardware muss Base + Offset addieren
- ALU Control ignoriert funct-Bits (*X*)
- Sendet hart verdrahtet ADD (0010) an die ALU

```
WENN ALUOp == 00:  
  Sende ADD an die ALU  
  (funct-Felder irrelevant)
```

Notizen

Adressberechnung: Augen zu für funct, immer Addition.

Fall 2: ALUOp = 01 (Branch)

- Chef funkt 01 — bedingter Sprung
- Zwei Register subtrahieren, um Zero zu füttern
- funct-Bits wieder ignoriert
- Sendet SUB (0110) an die ALU

```
WENN ALUOp == 01:  
  Sende SUB an die ALU  
  (Zero-Flag bei Gleichheit)
```

Notizen

Branch-Vergleich über Subtraktion; Zero-Ausgang meldet Gleichheit.

Fall 3: ALUOp = 10 (R-Type)

- Chef funkt 10 — Detail-Arbeit
- ALU Control erwacht und liest die funct-Felder
- Unterscheidung: add, sub, and, ...

```
Opcode 0110011, ALUOp = 10  
-> funct3 / funct7 scannen
```

Notizen

Bei R-Type muss die kleine Einheit funct3 und funct7 aktiv auswerten.

funct3 als Wegweiser

- Bei ALUOp = 10 ist funct3 der primäre Wegweiser
- funct3 = 111 — bitweises AND
- funct3 = 110 — bitweises OR
- funct3 = 000 — Arithmetik (Plus oder Minus?)
- Bei 000 reicht funct3 noch nicht — nächste Stufe

ALUOp	funct3	Aktion
10	111	AND
10	110	OR
10	000	Plus oder Minus? → Bit 30

Notizen

ADD und SUB teilen denselben funct3-Code. Ein Schritt tiefer ist nötig.

funct7 als letzter Schalter

- Nur wenn `ALUOp = 10` und `funct3 = 000`
- Relevant: Bit 30 der Instruktion (in `funct7`)
- Bit 30 = 0 — `add`
- Bit 30 = 1 — `sub`
- Rätsel entschlüsselt

```
add t0, t1, t2    # Bit 30 = 0  (funct7 = 0000000)
sub t0, t1, t2    # Bit 30 = 1  (funct7 = 0100000)
```

Notizen

Einziger physikalischer Unterschied zwischen `add` und `sub`: Bit 30.

Das 4-Bit-ALU-Kommando

- Ergebnis der ALU Control: 4 Steuerbits in die Haupt-ALU
- Standardcodierung nach Patterson/Hennessy
- ALU-Hardware schaltet stur bei diesen Bitmustern

4-Bit ALU-Kommando	Hardware-Reaktion
0000	bitweises AND
0001	bitweises OR
0010	ADDITION
0110	SUBTRAKTION

Notizen

Code `0010` aktiviert die Addier-Schaltung; `0110` invertiert B und subtrahiert.

Die Wahrheitstabelle der ALU Control

- Matrix der Experten-Box — als Gatter verlötet
- X rettet vor unnötigen Gattern bei Load/Store/Branch

ALUOp	Bit 30	funct3	Befehl	ALU-Output
00	X	X	Load/Store	0010 (Add)
01	X	X	Branch	0110 (Sub)
10	0	000	add	0010 (Add)
10	1	000	sub	0110 (Sub)
10	X	111	and	0000 (AND)

Notizen

Don't Cares verkleinern die obere Hälfte. Nur bei $ALUOp = 10$ werden funct-Bits exakt gelesen.

Warum zweistufig? (Effizienz)

- Monolith: 17 Bit Input permanent
- Box 1 (Main): 7 Bit $\rightarrow$ 128 Zeilen
- Box 2 (ALU): ca. 6 Bit $\rightarrow$ 64 Zeilen
- Zusammen unter 200 Zeilen statt ... 130.000
- Kunst der RISC-Hardware-Architektur



Notizen

Das 130.000-Zeilen-Monster wird in zwei winzige, schnell schaltende Tabellen zerlegt — Grundlage für hohen Takt und geringen Strom.

Die Architektur ist komplett

- Datenpfad gebaut (Einheit 5)
- Weichen und Enable-Pins analysiert
- Main Control spezifiziert
- ALU Control als Experte hinzugefügt
- CPU geistig und physikalisch vollständig — als Nächstes: Master Truth Table

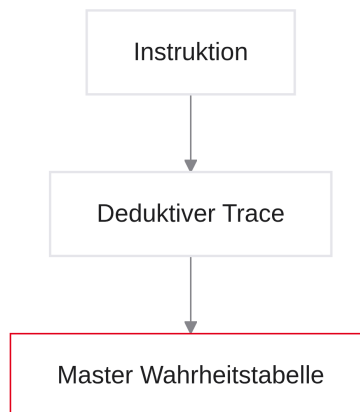
Notizen

Theoretische Konstruktion abgeschlossen. Als Nächstes füllen wir die Master-Tabelle für das Labor aus.

Signal-Trace (Die Meistertabelle)

Die Meistertabelle

- Alle Hebel in einer Matrix
- Deduktiv herleiten statt auswendig lernen
- Trace durch R-Type, Load, Store und Branch
- Don't Cares optimieren die Tabelle
- Referenzblatt für die Ripes-Laborsimulation



Notizen

Phase 5 beginnt: Matrix mit Stift und Papier füllen. Frage: „Wo fließen die Daten hin?“ Wer das herleiten kann, hat die Architektur verstanden.

Folie

60 / 84

Die finale Matrix (Das Skelett)

- Control-Tabelle nach Patterson/Hennessy
- Ausfüllen durch logisches Denken, nicht auswendig
- Vier Befehlsfamilien (Zeilen), acht Ausgangssignale (Spalten)
- Datenpfad aus Einheit 5 im geistigen Auge behalten

Instruk- tion	ALUSrc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
R-Type	?	?	?	?	?	?	?	?
lw	?	?	?	?	?	?	?	?
sw	?	?	?	?	?	?	?	?
beq	?	?	?	?	?	?	?	?

Notizen

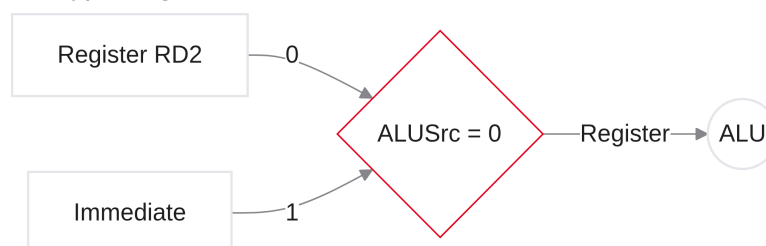
Leeres Arbeitsblatt: Kernstück der Rechnerarchitektur. Zeile für Zeile deduktiv schließen: Was der Befehl tut, bestimmt die Weichenstellungen.

Folie

61 / 84

Trace: R-Type (add) — ALUSrc

- Zeile für R-Type (Opcode 0110011)
- Womit rechnet die ALU? Mit dem zweiten Register ($rs2$)
- MUX vor der ALU schaltet das Register-Kabel durch
- $ALUSrc = 0$
- ImmGen bei R-Type abgeklemmt



Notizen

R-Type rechnet mit zwei Variablen aus der Werkbank, ohne Immediate. ALUSrc muss 0 sein, damit RD2 in die ALU fließt.

Folie

62/84

Trace: R-Type — Speichern

- Ergebnis verlässt die ALU und muss gespeichert werden
- Schreiben wir ein Register (`rd`)? Ja $\rightarrow$ `RegWrite` = 1
- Welches Signal speichern? Das ALU-Ergebnis
- MemtoReg klemmt den RAM ab $\rightarrow$ `MemtoReg` = 0

```
R-Type:
RegWrite = 1
MemtoReg = 0
```

Notizen

Bei `add` Ergebnis sichern: `RegWrite` = 1. Speichern wollen wir das Mathematik-Ergebnis, nicht RAM-Daten: `MemtoReg` = 0.

Folie

63/84

Trace: R-Type — RAM und ALUOp

- Benutzen wir den RAM? Nein $\rightarrow$ `MemRead` = 0, `MemWrite` = 0
- Bedingter Sprung? Nein $\rightarrow$ `Branch` = 0
- ALU-Kommando: Entscheidung delegieren $\rightarrow$ `ALUOp` = 10
- Erste Zeile der Meistertabelle ist komplett

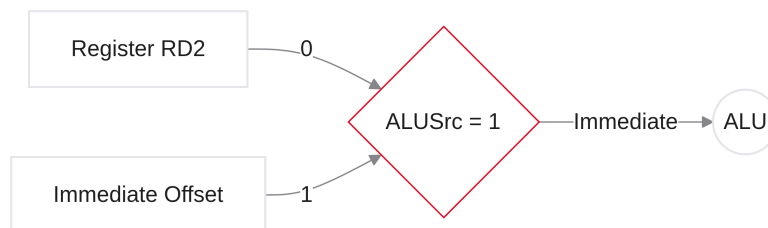
Instruk- tion	ALUSrc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
R-Type	0	0	1	0	0	0	1	0

Notizen

RAM unberührt, kein Sprung. Opcode verrät nicht `add` vs. `sub` — Funksignal 10 an die ALU Control.

Trace: I-Type (lw) — ALUSrc

- Load Word (Opcode 0000011)
- Adresse = Base + Offset → Immediate nötig
- ALUSrc schaltet ImmGen durch
- `ALUSrc = 1`



Notizen

Für die RAM-Hausnummer brauchen wir den Offset aus dem Befehlswort. Weiche auf 1: Immediate in die ALU.

Trace: I-Type (lw) — RAM

- Daten auslesen? Ja → `MemRead = 1`
- `MemWrite = 0` (Schutz)
- Letzte Weiche: RAM-Daten zum Register → `MemtoReg = 1`

```
lw:
MemRead  = 1
MemWrite = 0
MemtoReg = 1
```

Notizen

Lese-Befehl: MemRead auf 1, MemWrite auf 0. MemtoReg auf 1, damit der RAM-Inhalt (nicht die ALU) zum Register File geht.

Trace: I-Type (lw) — Reg und ALUOp

- In rd schreiben? Ja → `RegWrite` = 1
- Sprung? Nein → `Branch` = 0
- ALU muss Adresse addieren → `ALUOp` = 00
- Zweite Zeile steht

Instruk- tion	ALUSrc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
lw	1	1	1	1	0	0	0	0

Notizen

`RegWrite` = 1, `Branch` = 0. Main Control zwingt mit `ALUOp` 00 eine unbedingte Addition für die Adresse.

Trace: S-Type (sw)

- Store Word (Opcode 0100011)
- Adresse wieder Base + Offset → `ALUSrc` = 1
- In den RAM schreiben? Ja → `MemWrite` = 1
- `MemRead` = 0 (kein Kurzschluss)

```
sw:
ALUSrc    = 1
MemWrite  = 1
MemRead   = 0
```

Notizen

Wie Load am Anfang: `ALUSrc` = 1. Einziger Moment mit `MemWrite` = 1. `MemRead` bleibt 0.

Trace: S-Type (sw) — Don't Cares

- Werkbank-überschreiben? Nein $\rightarrow \text{RegWrite} = 0$
- MemtoReg bei geschlossener Tür wirkungslos $\rightarrow X$
- ALU wieder Adressaddition $\rightarrow \text{ALUOp} = 00$

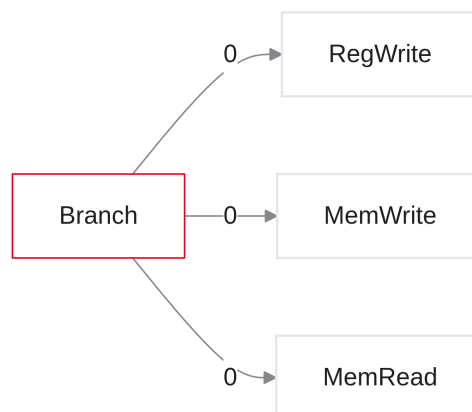
Instruk- tion	ALUSrc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
sw	1	X	0	0	1	0	0	0

Notizen

Store ändert keine CPU-Register. $\text{RegWrite} = 0$; $\text{MemtoReg} = X$. $\text{ALUOp} = 00$ für die Adresse.

Trace: B-Type (beq)

- Branch (Opcode 1100011)
- Vergleich zweier Register $\rightarrow \text{ALUSrc} = 0$
- Kein Speichern in RAM oder Register File
- $\text{MemWrite} = 0, \text{MemRead} = 0, \text{RegWrite} = 0$



Notizen

Zwei Register vergleichen: $\text{ALUSrc} = 0$. Branch ändert nur die Route — alle Enable-Pins auf 0.

Trace: B-Type (beq) — Control und X

- Branch-Weiche vor dem PC freigeben $\rightarrow \text{Branch} = 1$
- MemtoReg wieder wirkungslos ($\text{RegWrite} = 0$) $\rightarrow X$
- ALU: zwingende Subtraktion $\rightarrow \text{ALUOp} = 01$
- Meistertabelle vollständig deduziert

Instruk- tion	ALUSrc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
beq	0	X	0	0	0	1	0	1

Notizen

Branch = 1 fürs AND-Gatter. MemtoReg = X . ALUOp = 01 (SUB) für den Vergleich. Matrix komplett.

Die ausgefüllte Meistertabelle

- Referenz für Labor und Ripes
- X = Don't Care (Synthese-Freiheit)
- Ein Bitfehler in dieser Matrix kann Software still zerstören

Instruk- tion	ALUSrc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
R-Type	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Notizen

Kompass für das Praktikum: jede Zelle ist herleitbar, wenn Sie den Datenpfad verstehen.

Praxis: Signal-Analyse in Ripes

Vom Papier in den Simulator

- Theoretisches Konstrukt im echten Simulator testen
- „Paper & Pencil“: Signale vorhersagen, nicht blind klicken
- Control Unit visuell in Ripes finden
- Grüne und graue Steuerleitungen lesen
- Transfer: Steuersignale für unbekannte Befehle (z. B. `addi`) herleiten

Notizen

Die Blackbox ist entschlüsselt. Im Labor prüfen wir Bit für Bit, dass Ripes sich exakt wie unsere Matrix verhält.

Das Ziel im Labor

- Debugger: unerklärliche Registerzustände
- Ursache: ABI, Hardware, Tippfehler?
- Signalleitungen während des Clock-Ticks lesen
- Methode: Befehl ansehen → erwartete 1/0 → Linie prüfen

Ein Ingenieur klickt nicht auf Run und hofft.
Ein Ingenieur formuliert eine These und verifiziert sie.

Notizen

Wissenschaftlicher Ansatz: Hypothese formulieren („MemWrite muss aufleuchten“), Clock, verifizieren.
Dunkel bleibendes Signal zeigt oft den Bug.

Wo ist die Control Unit in Ripes?

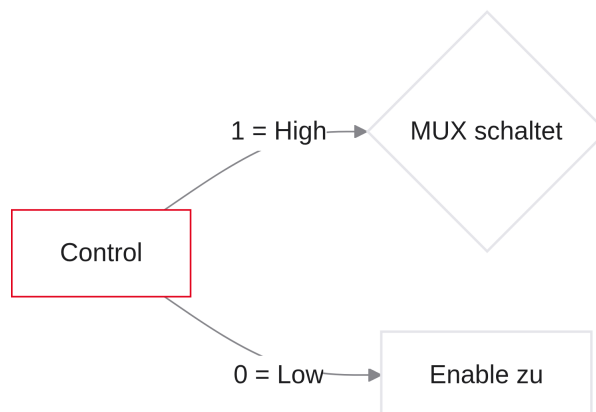
- Tab Processor öffnen
- Mittig oben: abgerundeter Block Control
- Instruction-Leitung von unten links (Opcode-Input)
- Rechts: einzelne Steuerfäden aus dem Block
- Signalnamen manchmal abgekürzt — Konzept identisch

Notizen

32-Bit-Instruktionskabel hinein; 1-Bit-Steuerkabel wie Spinnenweben nach rechts — live beobachten.

Grüne vs. graue Leitungen

- Ripes-Farbkonzept für 1-Bit-Steuerleitungen (aktuelle Versionen)
- Grün: logische 1 (High)
- Grau: logische 0 (Low)
- Mauszeiger: Bitwert als Text-Popup
- Don't Care (X) erscheint **nicht** als eigener Anzeigezustand — der Simulator zeigt immer 0 oder 1



Notizen

Grün = aktiv. Grau = inaktiv. X ist nur Tabellenkonzept: bei $\text{RegWrite} = 0$ ist der angezeigte MUX-Wert egal.

Labor-Methode: Paper & Pencil

- Drei mentale Vorhersagen vor dem Klick
- Übung 1 (sub): Wie leuchten RegWrite und MemWrite?
- RegWrite grün (1) — ALU-Ergebnis speichern
- MemWrite grau (0) — RAM unberührt

```
sub t0, t1, t2
# These: RegWrite=1, MemWrite=0
# Clock in Ripes -> These prüfen
```

Notizen

Training: bei sub speichern wir, schreiben aber nicht in den RAM. Im Praktikum mit Clock verifizieren.

Vorhersage-Übung: lw

- Übung 2 (lw): Stellung von ALUSrc und MemtoReg?
- Beide MUX-Steuerungen auf 1 (grün)
- ALUSrc = 1: Offset in die ALU
- MemtoReg = 1: RAM-Daten ins Register

```
lw t0, 4(s1)
# These: ALUSrc=1, MemtoReg=1
```

Notizen

Load: Konstante für Adresse und RAM-Pfad für Write-Back — beide Steuerleitungen grün.

Vorhersage-Übung: bne

- Übung 3 (bne): Branch und ALUOp?
- Branch = 1 (grün) — Sprungfamilie aktiv
- Main Control funkt 01 → ALU Control erzwingt SUB
- PCSrc klappt nur, wenn die Branch-Bedingung wirklich gilt

```
bne t0, t1, Label
# These: Branch=1, ALU=SUB
# PCSrc nur bei genommener Bedingung
```

Notizen

Bei `bne`: Branch grün, ALU subtrahiert. Sprung nur, wenn Zero die Ungleichheit bestätigt (typisch: Branch AND NOT Zero). Gleichheit — AND bleibt 0, PCSrc blockiert.

Folie

80 / 84

Fehleranalyse: Typo in der Matrix

- Was, wenn in der Tabelle bei `add` versehentlich `MemWrite = 1` stünde?
- Addition nach `t0` klappt vordergründig
- Parallel: ALU-Ergebnis als RAM-Adresse, `t2` am Write-Data-Port
- Zufällige RAM-Zelle wird zerstört — Bug oft erst viel später sichtbar

Notizen

Klassischer Albtraum: stiller Speicherschaden. Deshalb Verifikation der Wahrheitstabelle in der Industrie so kritisch.

Folie

81 / 84

Labor-Aufgabe 1: Transferleistung

- Bisher nur reine Basis-Befehle
- Aufgabe: Signale für `addi` (Add Immediate) herleiten
- Tipp: liest Konstante (wie `lw`), speichert ALU-Ergebnis (wie `add`)
- Auf Papier ausfüllen, dann in Ripes an den Kabeln prüfen

Instruk- tion	ALUSrc	Mem- toReg	Reg- Write	Mem- Read	MemWrite	Branch	ALU- Op1	ALU- Op0
<code>addi</code>	?	?	?	?	?	?	?	?

Notizen

Transferleistung: ADDI kombiniert ImmGen-Pfad und Register-Write-Back. Tabelle selbst ableiten und am Simulator verifizieren.

Verifikation in Ripes

- Assembler im Editor-Tab eingeben
- Zum Processor-Tab wechseln
- Clock, bis der Befehl die Control-Phase erreicht
- Maus über grüne/graue Linien aus der Control-Box
- Werte mit Ihrer Tabelle vergleichen

```
Gruenes Kabel am ALUSrc-MUX -> 1 in der Tabelle  
Graues Kabel                -> 0 in der Tabelle
```

Notizen

Workflow: tippen, Tab, Clock, Kabel ablesen. Grün = 1 in der Tabelle, grau = 0.

Labor-Aufgabe 2: Der MUX-Beobachter

- Mini-Programm (4 Zeilen):
- `li t0, 5 - add t1, t0, t0 - sw t1, 0(s1) - beq t0, t1, Lbl`
- Mit Clock durchticken
- Augen nur auf den MemtoReg-MUX
- Protokoll: Status (0, 1, X) in Takt 1–4

```
Fokus auf ein einziges Bauteil hilft gegen  
Reizueberflutung im Diagramm.
```

Notizen

Gegen Reizüberflutung: ein Bauteil fokussieren. Beobachten Sie, was Ripes bei Don't Care zeigt.

Q&A und Hands-On

- Ripes starten
- Meistertabelle als Kompass nutzen
- Unerwartetes Verhalten: Logik der Hardware ergründen
- Viel Erfolg bei der Entschlüsselung des Steuerwerks



Notizen

Sie beherrschen Datenpfad und Signallogik. Ripes zementiert das Wissen. Fragen willkommen — viel Erfolg beim Hardware-Debugging.